```
==========================================================
======
```

User defined command

User defined commands

Commands

```
:help command
:help command-nargs          " nargs attribute
:help command-register       " replacement help

"" command [{cmd}]
" List all user defined commands
:command                     " All commands
:command Syntastic           " Begins with 'Syntastic'


"" command[!] [{attr}…] {cmd} {rep}
" Define a user command
"  {attr}: command attributes
"  {cmd}: command name
"  {rep}: replacement text
:command -nargs=0 MyCommand call MyFunction()
 " :MyCommand is the same as :call MyFunction()
:command! …          " replace existing command
```

Attributes

```
-nags=0          " No arguments (default)
-nargs=1         " Exactly 1 argument
-nargs=2         " ERROR. Only 0/1 are supported.

-nargs=*         " zero or more
-nargs=?         " 0 or 1
-nargs=+         " 1 or more
```

Replacement

```
<line1>
<line2>
<bang>            " ! if command was invoked with a !.
Nothing otherwise

<args>
<q-args>          " Quotes the arguments
                  " e.g. a b c  =>  "a b c"
<f-args>          " Use as function argument. Arguments
separated as spaces.
                  " e.g. a b c  ==> "a","b","c"


e.g.
:command -nargs=* MyCommand call MyFunction(<f-args>)
:MyCommand 1 2          " Same as :call
MyFunction("1","2")
```

Misc
*) Use whitespace in <f-args>
Use '\' to include white space

```
a b       "a","b"
a\b       "a\b"
a\ b      "a b"
a\\b      "a\\b"
a\\\ b    "a\ b"
```

==========

Undo

undo branch: vim keeps all undo history in a tree.
change number:
Use :undolist or undotree() to see change numbers.

Mappings

```
u         Undo
U
r         Redo

g-        :earlier
g+        :later
```

Options
Undofile option
Allows undos to be saved.
- saves undos on file write
- reloads saved undos on file read

```
" undofile
set undofile
set noundofile

set undodir          " directory where undofiles are
stored.
```

Commands and functions

```
"" undo/redo
:undo
:undo 10            " Undo to change #10
:redo

" Relative undo
:earlier 1          " 1 change before
:earlier 2s         " 2 seconds
:earlier 2m         " minutes
:earlier 2h         " hours
:earlier 2f         " file saves

:later 1            " 2 changes later
```

```
:later 2s


"" undo branch
"
" :undolist
" Shows undo tree leaves
:undolist
e.g. sample output

# number = undo number
# changes = number of changes from root
# when = time of change
# saved = save number
number changes  when               saved
     1       1  07:22:55
    14      13  07:23:45             4
    56       3  07:45:52            25

" See full undo tree
echo undotree()


"" Read/write
wundo ./myundos
rundo ./myundos
```

==========

Spelling

spellfile: File where good/bad spellings are stored
internal-wordlist: List where temporary good/bad spellings are stored. Lost when vim is exited.

Help

```
:help spell
```

Options

```
"" spell
" Toggle spelling check
:set spell            Enable
:set nospell          Disable

"" spelllang
" Set spelling language
:set spelllang=en.utf-8


"" spellfile
" File where marked good/bad words are stored
" Default is empty
" File name must be of the form
"   {encoding}.add
:set spellfile=~/.vim/myspell/en.utf-8.add
```

```
"" spellbadword()
" List of bad words in file
call spellbadword()
```

Keys

```
]s          Next misspelled word.
[s          Previous.
]S          Same as ]s, except only bad words. i.e.
ignore rare or other local words
[S          Same as [s, except only bad words.

zg          Mark word as good word, in global spellfile
zG          Mark word as good word in internal word
list. Lost on exit.
zw          Mark word as bad word in global spellfile
zW          Mark word as bad word in internal word list

zuw         Undo marking
zug
zuW
zuG

z=          Suggest correct spelling
```

```
 ========================================================
                            ======
                        Auto command

:help autocommand
:help autocommand-events
:help autocommand-patterns
```

Autocommand group (augroup)
Group for a set of commands
Useful for modifying/removing a set of commands
e.g. remove all commands for group "myplugin"

Basic

```
"" Define command
:au BufRead * echo "Hello"               " event=BufRead
pattern=* command=echo "Hello"
:au BufRead end …                    " default group
:au BufRead END …                    " same


" patterns:
" If there is a / in the pattern, then it checks the
full path.
" Otherwise, it just checks the value.
:au BufRead *.txt echo "Hello"          " .txt in file
name
:au BufRead /*.txt echo "Hello"         " .txt files in
file full path.
```

```
:au BufRead *.txt,*.js echo "Hello"     " Multiple
:au FileType python echo "Hello"        "  'filetype'
option with value "python"
:au VimEnter *


"" Remove commands
:autocommand !            " Remove all
:au!                      " same
:au! BufRead *.txt        " Remove by pattern
:au! BufRead              " Remove by event i.e. all
commands for event
:au! mygroup              " Remove by augroup=mygroup
:au! mygroup BufRead
:au! mygroup BufRead *.txt


"" Grouping commands
" aug: sets the current group
" group end or END to set default
:augroup            " list existing groups
:aug
:aug mygroup        " use group mygroup
  :au BufRead …     " command in mygroup
  :au BufRead …
:aug END            "
:aug end            " same

" Remove group
:aug! mygroup       " remove group test
```

Events

```
:help autocommand-events
BufNewFile     " Start editing a new file.
BufReadPre     " Editing an existing file, before read.
Not invoked on new files.
BufRead        " After read.
BufReadPost    " Same

BufEnter       " After entering a buffer
BufLeave       " Before leaving a buffer

BufNew         " Immediately after creating a new
buffer.


BufWrite        " Start to write i.e. save
BufWritePre     " Same
BufWritePost    " After write
BufWriteCmd

FileType       " When 'filetype' option is set
Syntax         " When 'syntax' option is set
```

Order of events per action

Action) Opening a new buffer
BufEnter

Action) Open an existing file
BufNew
BufEnter
BufRead

Action) Saving

Patterns

```
*               any sequence of characters
?               any single character
[ab]            'a' or 'b'
[^ab]           not 'a' or 'b'
,               multiple patterns
                e.g. *.json,*.js
\,              ',' character.

*.txt               If pattern doesn't begin with '/',
match against file name only
/tmp/*/*.txt        If pattern begins with '/', match
against
~/*.txt             '~' = $HOME
```